CS1910 Computer Science I • Tuples and Comprehensions **Partner workbook**

Images as 2D Lists

In lecture with a partner • about 35 minutes

Name: _____ **Partner:** _____

This PDF can be filled in: click in an answer space to type, or print it and write. Draw diagrams with your PDF viewer's pen or markup tool, or on paper.

What this workbook practises

A grayscale image can be stored as a **2D list**: a list of rows, where each row is a list of pixel values from 0 (black) to 255 (white). You will build images with nested loops, look for patterns in the numbers, generalize with a function, and then practise list comprehensions.

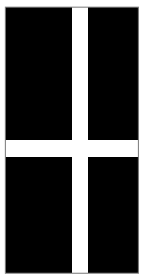
```
# a 3 x 3 image
image = [[pixel, pixel, pixel],
         [pixel, pixel, pixel],
         [pixel, pixel, pixel]]
```

1 The four-quadrant image

about 12 minutes

The problem

Create a **128** × **64** image (height 128 rows, width 64 columns) that is black, with a white line across the middle horizontally and vertically.



A smaller 16 × 8 version of the same idea. Each square is one pixel: 0 is black and 255 is white.

1. **Abstraction:** what details matter, and what can we ignore? If you did not know this was an image, what patterns could you still find in the numbers?

2. The image has height 128 (rows 0 to 127) and width 64 (columns 0 to 63). Which row and which column are "the middle"?

3. The 2D list could hold 128 lists of 64 values, or 64 lists of 128 values. Which one is a "list of rows"?

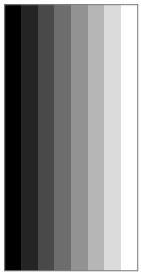
4. Write the code to build `image`.

2 Gradient images

about 12 minutes

The problem

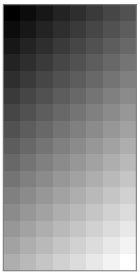
Create a 128 × 64 **gradient** image that fades from black to white (a) from left to right, and (b) from top to bottom.



(a) left to right



(b) top to bottom



(c) corner to corner

5. What stays the same as the four-quadrant problem? What needs to change?

6. If a pixel is no longer just black or white, what decides its value? Which index controls it in (a)? In (b)?

7. **Generalization:** can one function create both images (and the corner-to-corner one)? What would its parameters be?

8. Write `gradient(height, width, direction)`, which returns the 2D list.

3 Seven list comprehensions

about 10 minutes

A list comprehension has the form `[expression for item in iterable if condition]`. Before writing each one, say *what data it works over* and *what condition (if any) limits it*.

```
numbers = range(100)
words = ["lower", "case", "version", "words"]
number_strings = ["1", "123", "456", "789"]
strings = ["a", "aa", "aaa", "aaaa"]
grades = [49, 47, 23, 32, 44, 52]          # marks out of 53
floats = [3.14, 22/7, 355/113, 3.0, 25/8]
```

#	Task	Comprehension
1	the numbers in 0–99 divisible by 7	
2	the upper-case version of `words`	
3	`number_strings` as `int`s	
4	square roots of the numbers in 0–999 divisible by 77	
5	the lengths of `strings`	
6	each grade as a percentage of 53	
7	how close each float is to π	

✓ Before you leave

about 1 minutes

- ☐ I can build a 2D list of rows with nested loops, and index it as `image[row][col]`.
- ☐ I can work out a pixel's value from its row and column.
- ☐ I can generalize two similar programs into one function with a parameter.
- ☐ I can write a list comprehension with an expression and a condition.